Evaluation 1: Baseline and Problem Understanding

Measuring the Systems Cost of Heterogeneous KV Quantization

Shivam Patel (2025202030) · Suyash Pande (2025201063) · Divyanshu Jain (2025201059)

CS3.406 Programming AI Accelerators · Plan as of 17 October 2026

1. Goal of Evaluation 1

Evaluation 1 (week 5, 20% of the grade) presents baseline results and the plan they lead to. We do not build the custom kernel yet. We answer two questions with measurements on Qwen2.5-7B-Instruct:

1. **Which KV heads matter, and does it depend on the task?** We profile per-head attention behaviour and per-head quantization sensitivity across three tasks.
2. **Does KV quantization reduce or increase the cost of inference?** We measure throughput, latency and memory before and after uniform KV quantization, both in systems that dequantize naively and in systems with fused kernels.

The project question is unchanged from the proposal: is per-head mixed-precision KV free at inference time, or do dispatch, load imbalance and unpacking costs eat the savings? Evaluation 1 gives us the sensitivity ranking that sets the bit-widths, and the cost baseline that any mixed-precision kernel must beat.

The course rubric asks Evaluation 1 to show a working baseline, reliable measurements, initial profiling, an identified bottleneck and a proposed optimization strategy. Its key question is: **what exactly are we trying to improve, and how do we know it is the bottleneck?**

2. Hardware and Software

The college now gives us remote access to an **NVIDIA RTX PRO 5000 (Blackwell, 48 GB)**. This replaces the 24 GB target in the proposal, so we can run a full 7–8B model at long context and drop the 3B model.

Item	Value
GPU	NVIDIA RTX PRO 5000, Blackwell architecture (sm_120)
Memory	48 GB
Memory bandwidth	About 1.3 TB/s (approximate; we will measure it ourselves)
CUDA	12.8 or later (required for Blackwell)
PyTorch	2.7 or later, cu128 build
Profilers	Nsight Systems, Nsight Compute, nvidia-smi, NVML
Libraries	HuggingFace transformers, optimum-quanto, vLLM; LMDeploy if a Blackwell build exists

Week 1 checks:

- ☐ Install the cu128 PyTorch build. Older wheels fail on sm_120 with "no kernel image available".
- ☐ Confirm `ncu` can read performance counters (no `ERR_NVGPUCTRPERM`).

- ☐ Ask whether we can lock clocks (`nvidia-smi -lgc`) and get exclusive access during measurement runs.
- ☐ Confirm vLLM (and LMDeploy) run on this GPU before relying on them as baselines.
- ☐ Measure real peak DRAM bandwidth with a copy kernel, so roofline numbers use our own figure.

3. Model

We use **Qwen2.5-7B-Instruct** for Evaluation 1. Access to Llama-3.1-8B-Instruct is pending; when it arrives, Llama becomes the second model family, and comparing the two is itself an analysis point.

Property	Qwen2.5-7B-Instruct	Llama-3.1-8B-Instruct (later)
Layers	28	32
Query heads per layer	28	32
KV heads per layer	4	8
Query heads sharing one KV head	7	4
Head dimension	128	128
Weights in BF16	About 15 GB	About 16 GB
KV cache per token (FP16)	56 KB	128 KB
KV cache at 32K context	1.75 GB	4 GB
Native context	32K (128K with YaRN)	128K
KV heads to rank (quantization units)	28 × 4 = 112	32 × 8 = 256

KV cache per token = 2 (K and V) × layers × KV heads × head dim × 2 bytes. For Qwen: 2 × 28 × 4 × 128 × 2 = 57,344 bytes = 56 KB.

Why Qwen works well here:

- It fits easily: 15 GB of weights plus 1.75 GB of KV per 32K sequence leaves room for batch 8 at 32K (about 29 GB total) on 48 GB.
- Its keys contain large outlier channels, so it is known to be **more sensitive to KV quantization** than Llama. Differences between heads should be easy to see.
- It needs no access approval.

What a "head" means for us. With grouped-query attention, 7 query heads share each KV head. The KV cache stores KV heads, so quantization happens per KV head. We therefore work at two granularities:

- **Attention behaviour** is recorded per query head (28 × 28 = 784 heads), because that is where attention patterns live.
- **Sensitivity and bit-width** are assigned per KV head (112 units), because that is what can be stored at a different precision.

4. Tasks and Datasets

We use three tasks because head importance depends on the task. Published work shows that a small set of "retrieval heads" does almost all long-context copying (Retrieval Head, Wu et al., 2024), and that heads split into retrieval and streaming types (DuoAttention, 2024). A plain-text

calibration set would rank those heads as unimportant; quantizing them hard would keep perplexity fine but break retrieval.

Task	Dataset	Context	Heads expected to matter	Quality metric
Language modeling	WikiText-2, PG19	4K, 8K, 16K, 32K	Local and general heads	Perplexity
Long-context retrieval	Needle-in-a-haystack / passkey (we generate it)	4K–32K, several needle depths	Retrieval heads (few, critical)	Retrieval accuracy
Multi-step reasoning	GSM8K with chain-of-thought	Short prompt, long generation	Heads used during decode; errors build up over many tokens	Exact-match accuracy
Long-document QA (optional)	LongBench subset (HotpotQA or code completion)	8K–16K	Multi-hop and copying heads	F1 / edit similarity

Rules to keep the results honest:

- **Calibrate on train splits, evaluate on test splits.** Head rankings come from PG19 train, our own generated needles and GSM8K train. Results are reported on WikiText-2 test, PG19 test, new needle depths and lengths, and GSM8K test.
- **Compare at equal cost.** Quality is plotted against average bits per KV value, including scale and zero-point overhead, not against scheme names.
- **Score each task separately before combining**, so we can show whether the rankings agree.

5. Analysis 1: Which Heads Activate

We run each task through the FP16 model and record how each query head distributes its attention. This shows what each head does; Section 6 tests whether that matters for quantization.

Statistic	Definition	What it identifies
Attention entropy	Entropy of each head's attention distribution, averaged over query positions	Focused heads (low) vs. diffuse heads (high)
Sink fraction	Share of attention placed on the first token	"Attention sink" heads that do little real work
Local fraction	Share of attention within the last 64 tokens	Local / streaming heads
Retrieval score	On needle prompts: how often a head's top-attended token is the needle token being copied (Wu et al., 2024)	Retrieval heads
Key outlier ratio	Largest $\|K\|$ channel magnitude divided by the median, per KV head	Heads whose keys are hardest to quantize

Implementation notes:

- A full attention matrix at 32K is 28 heads × 32K × 32K per layer, too large to store. We capture Q and K with forward hooks and compute softmax only for a sample of query positions (for example the last 128), which also matches what decoding does.

- Keys are captured **after RoPE**, since that is what the KV cache stores and what will be quantized.
- Output: one layer × head heatmap per statistic per task.

6. Analysis 2: Which KV Heads Are Sensitive

We quantize **one KV head at a time** and measure how much the model's output changes. The result is a ranking of all 112 KV heads for each task.

Method:

1. Register a custom attention function in transformers that wraps SDPA and fake-quantizes (quantize then dequantize) the post-RoPE K and V of one chosen KV head. Doing this inside attention means **prefill also sees the quantized values**; a normal single forward pass would otherwise run all attention at FP16 and never test the quantized cache.
2. Use **2-bit** for the ranking sweep, since 4-bit errors may be too small to separate heads. Re-check the top-ranked heads at 4-bit.
3. Score each head by the **KL divergence** between the FP16 model's next-token distribution and the perturbed one, averaged over tokens. KL needs no labels and is more sensitive than loss. For needle prompts we also record the retrieval-accuracy drop. For GSM8K we use KL on teacher-forced reference solutions, because generating per head is too slow.
4. Repeat for each task: 112 heads × 3 tasks, with about 16 samples per task. This is a few thousand forward passes, roughly 1–2 GPU-hours on this GPU (estimate, to be confirmed with a timing run).

Analysis of the rankings:

- **Task dependence:** Spearman rank correlation between the rankings of each pair of tasks. Low correlation is direct evidence that sensitivity depends on the task.
- **Activation vs. sensitivity:** correlation between Section 5 statistics (retrieval score, key outlier ratio) and sensitivity. A head that is active is not necessarily sensitive, and this tests it.
- **Combined ranking:** normalize each task's scores and take the maximum per head, so a head that matters for any task is protected.
- **Bit allocation:** greedily assign bits from {2, 4, 8} to KV heads under an average-bit budget (for example 3 and 4 bits), most sensitive heads first.
- **Validation:** compare the mixed allocation against uniform quantization at the same average bits on the held-out test sets.

7. Baseline Cost: Before and After Quantization

How quantization is implemented decides whether cost goes up or down. We measure uniform KV quantization against the FP16 baseline in these systems:

System	KV format	How attention reads it	Expected result
HuggingFace transformers, default cache	FP16	Directly	Baseline
HuggingFace QuantizedCache (quanto backend)	INT4 and INT2 stored	Dequantized back to FP16 before attention	Memory down, latency up
vLLM with `kv_cache_dtype="fp8"`	FP8	Fused kernel reads FP8 directly	Memory down, latency down at long

System	KV format	How attention reads it	Expected result
			context
LMDeploy TurboMind (if it runs on Blackwell)	INT8 / INT4 (`quant_policy` 8 / 4)	Fused kernel with in-register dequantization	Memory down, latency down

Sweep: context length 4K, 8K, 16K, 32K × batch size 1, 4, 8, for each system.

Metric	How we measure it
Time to first token (TTFT)	CUDA events around prefill
Time per output token (TPOT)	CUDA events per decode step, median over steps
Throughput	Generated tokens per second, per batch size
Peak GPU memory	`torch.cuda.max_memory_allocated` plus nvidia-smi for engines that pre-allocate
KV cache size	Computed bytes and measured allocation
Maximum batch at 32K	Largest batch that fits on 48 GB, FP16 vs. quantized
Quality	The three tasks from Section 4
Energy per token (optional)	NVML cumulative energy counter over a long decode loop

Profiling to identify the bottleneck (Nsight):

- **Nsight Systems:** share of each decode step spent in attention vs. linear layers, as context grows. This shows where KV reads start to dominate.
- **Nsight Compute on the decode attention kernel:** DRAM throughput as a percentage of peak (`dram__throughput`) and bytes read (`dram__bytes_read.sum`). Near-peak throughput proves the kernel is bandwidth-bound, so reducing bytes is the lever.
- **Roofline:** place the FP16 and quantized attention kernels on a roofline using our measured bandwidth.
- **Naive path:** in the quanto path, find the separate dequantize kernels and the extra memory traffic they cause.

8. Measurement Protocol

Every number we report follows the same procedure:

1. Persistence mode on; clocks locked with `nvidia-smi -lgc` where allowed.
2. 20 warmup iterations discarded, then 100 timed iterations.
3. Median reported with interquartile range, never a single run or a bare mean.
4. Run-to-run variance reported next to every speedup claim.
5. If clocks cannot be locked, or the GPU is shared, we report the variance and state the limitation.
6. Correctness first: outputs compared against an FP32 reference with a relative tolerance before any timing is reported.

9. Expected Observations and Plan After Evaluation 1

These are hypotheses that the Evaluation 1 measurements will confirm or reject:

Hypothesis	Evidence that tests it
Decode is bandwidth-bound and attention's share of each step grows with context	Nsight Systems breakdown; Nsight Compute DRAM throughput near peak
A few heads (early layers, retrieval heads) are far more sensitive than the rest	Long-tailed sensitivity distribution; heatmaps
Sensitivity rankings differ by task	Low Spearman correlation between tasks
Naive quantized cache saves memory but increases latency	quanto vs. FP16 TPOT and memory
Fused kernels save both memory and time at long context	vLLM FP8 / LMDeploy vs. FP16
No existing system supports a different bit-width per head	Survey of system options; this is the gap we fill

Plan after Evaluation 1 (follows the proposal's critical path):

Week	Work	Milestone
6	FP16 paged split-K decode kernel, validated against SDPA; `quantize_pack` kernel with INT4/INT8 packing and fused unpacking	Uniform-precision kernel works
7	Evaluation 2: end-to-end generation with our kernel, comparison with the Section 7 baselines, first quality numbers	Working prototype
8	Per-head mixed precision using the Section 6 allocation. Dispatch strategies compared. Llama as second model family	Cost of heterogeneity measured
9	Final evaluation: capacity demo, ablations, report	Complete

10. Open Issues From the Proposal Review

Reviewing the proposal raised points to fix before or during the kernel work:

Kernel design

- **Runtime branching may show no divergence.** If each warp handles one KV head, a per-head bit-width branch is the same for the whole warp. The real costs of runtime dispatch are register pressure (registers sized for the FP16 path), load imbalance between blocks and instruction-cache size. We will measure occupancy, registers per thread and tail idle time, and use the divergence metric to confirm.
- **Launch overhead of templated kernels can be removed** with CUDA Graphs. We add a third strategy: one launch where each thread block branches on `blockIdx` (a grouped launch).
- **Load balancing:** split-K divides work by token count, but INT4 heads read a quarter of the bytes of FP16 heads. We will split by bytes so blocks finish together.
- **Page layout** for heads with different byte sizes (separate pools per bit-width, or per-head page tables) must be decided.

Quantization scheme

- The proposal says both "per-token" and "one scale per page". We must pick one.

- KIVI shows keys need **per-channel** quantization (outlier channels) while values can be per-token. This matters even more for Qwen.
- New tokens during decode need an FP16 buffer until a group is full (as KIVI does).
- Scale and zero-point add about 6% at INT4 (about 4.25 effective bits); this goes into the "average bits" axis.

Evaluation

- Decode attention's work per byte does not change with batch size, so the "crossover" should be stated over context length, not batch size. There may be no crossover at all, and that is a valid result.
- Energy: use the NVML cumulative energy counter, not instantaneous power samples, which are too coarse for short kernels.
- Perplexity at 32K: compute the loss in chunks; full-vocabulary logits for 32K tokens do not fit in memory.

Proposal document

- Add a **Risks / Fallback** section (required by the guidelines), including a hardware fallback.
- Fill in the **team split** (Section 8.1 is empty).
- Fix the statement that profiling is 20% of the grade (Evaluation 1 is 20% and covers more than profiling).
- Cite which papers claim mixed precision is free, and soften "first real measurement".
- Remove the repeated tables in Section 7; update hardware from the 24 GB target to the RTX PRO 5000.

11. Deliverables and Task Split

What we present at Evaluation 1:

- ☐ Platform set up and verified (Section 2 checks), with measured peak bandwidth.
- ☐ Attention-statistics heatmaps per task (Section 5).
- ☐ Per-KV-head sensitivity ranking per task, rank correlations and a proposed bit allocation (Section 6).
- ☐ Before/after cost table and plots: TTFT, TPOT, throughput, memory, max batch (Section 7).
- ☐ Nsight evidence that decode attention is bandwidth-bound, with a roofline plot.
- ☐ Quality numbers for FP16 and uniform quantization on the three tasks.
- ☐ Plan of action for weeks 6–9, based on these observations (Section 9).

Workstreams (owners to be agreed by the team; each is independently assessable):

Workstream	Scope	Owner
Head analysis	Attention statistics hooks, per-KV-head sensitivity sweep, rankings and bit allocation (Sections 5–6)	
Systems baseline	Cost benchmarks across HF / quanto / vLLM / LMDeploy, Nsight profiling, roofline (Section 7)	
Tasks and quality	Datasets, needle generator, evaluation harness, quality metrics, measurement protocol (Sections 4, 8)	